AWS Java Login Web Application Using Amazon RDS

1. Introduction

The objective of this project is to deploy a Java Login and Registration Web Application on Amazon Web Services (AWS). The application uses an Amazon EC2 instance as the application server and Amazon RDS for MySQL as the database service.

The project demonstrates cloud infrastructure configuration, networking, server management, database connectivity, application deployment, and testing.

The application was built using Maven, deployed on Apache Tomcat, and accessed through a web browser. A private virtual machine was used for database administration and to connect to Amazon RDS.

AWS Region: Asia Pacific (Mumbai) – ap-south-1

2. Project Objectives

- Create and configure a custom Amazon VPC.
- Configure public and private subnets.
- Configure routing and network connectivity.
- Deploy an EC2 application server and a private virtual machine.
- Create an Amazon RDS for MySQL database.
- Configure security groups for controlled database access.
- Install Java, Maven, Git, and Apache Tomcat.
- Build and deploy the Java web application.
- Connect the application to Amazon RDS.
- Test registration, login, and database operations.

3. AWS Infrastructure Setup

3.1 Creating the VPC

I created a custom Amazon Virtual Private Cloud (VPC) named java-lab-vpc.

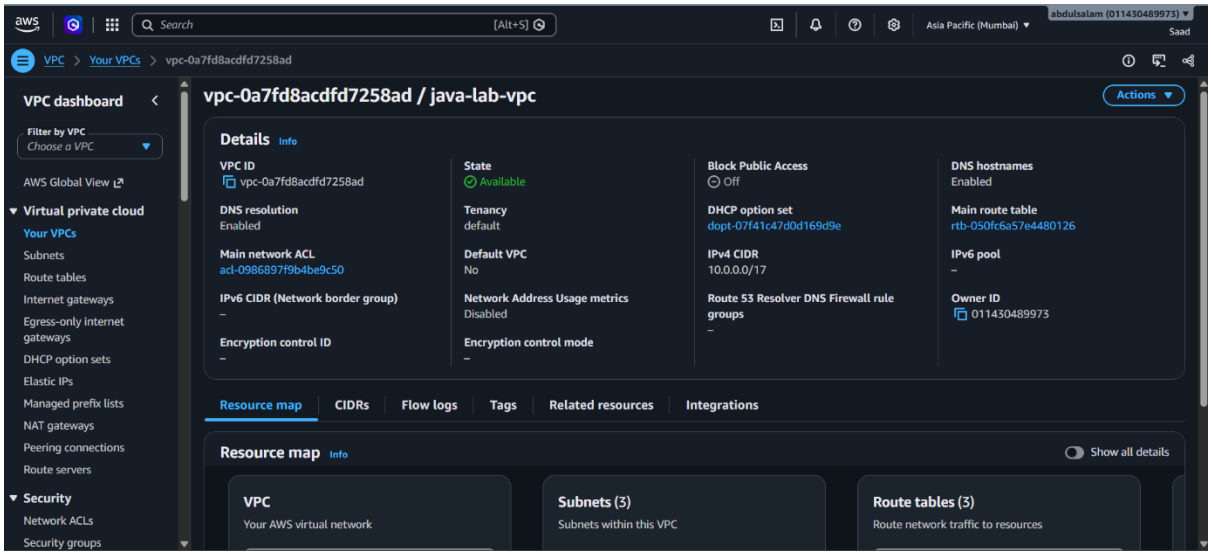
The VPC provides an isolated virtual network for the application's AWS resources.

Configuration:

- VPC Name: java-lab-vpc
- AWS Region: ap-south-1
- IPv4 CIDR: 10.0.0.0/17

Procedure:

- Opened the AWS Management Console.
- Navigated to the VPC service.
- Selected Your VPCs and clicked Create VPC.
- Selected the VPC-only option.
- Entered the VPC name and IPv4 CIDR.
- Created the VPC.
- Verified that the VPC was available.



3.2 Creating the Subnets

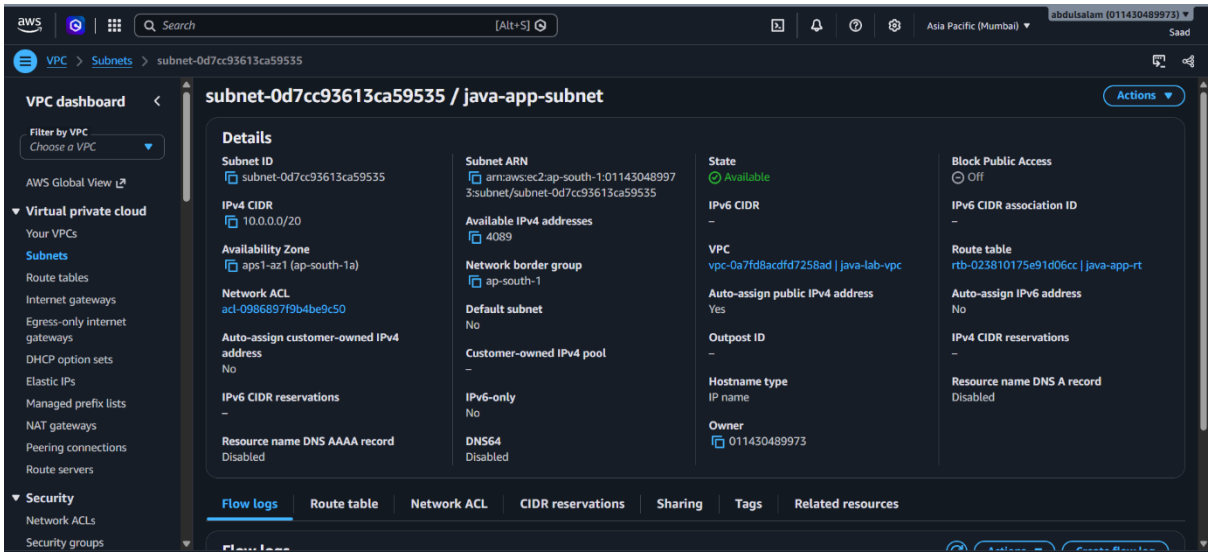
I created subnets within java-lab-vpc to separate application and private resources.

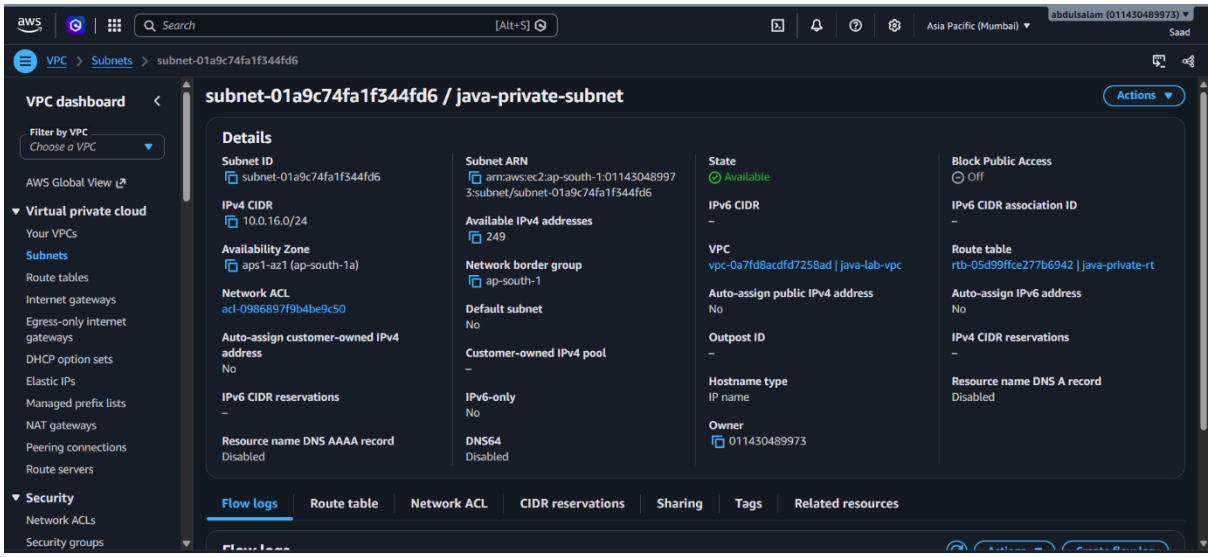
The recorded subnet configuration is:

Subnet	CIDR
java-app-subnet	10.0.0.0/20
java-private-subnet	10.0.16.0/24

Procedure:

- Opened VPC → Subnets.
- Selected Create subnet.
- Selected java-lab-vpc.
- Created the application subnet using the recorded CIDR.
- Created the private subnets using their respective CIDRs.
- Selected the appropriate Availability Zones according to the lab configuration.
- Verified the subnet names, CIDRs, and VPC association.





3.3 Configuring Internet Gateway and Routing

I configured network routing to provide the required connectivity to the application and private resources.

Procedure:

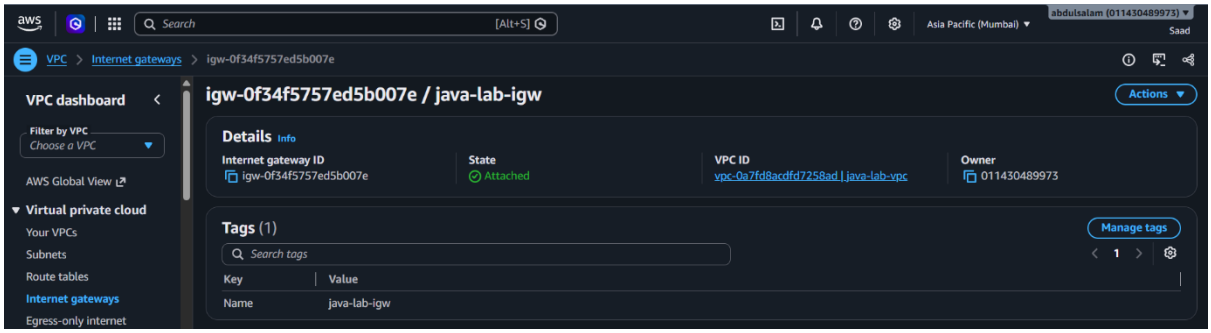
- Created and attached an Internet Gateway to the VPC.
- Configured the public route table.
- Added the default route 0.0.0.0/0 pointing to the Internet Gateway.

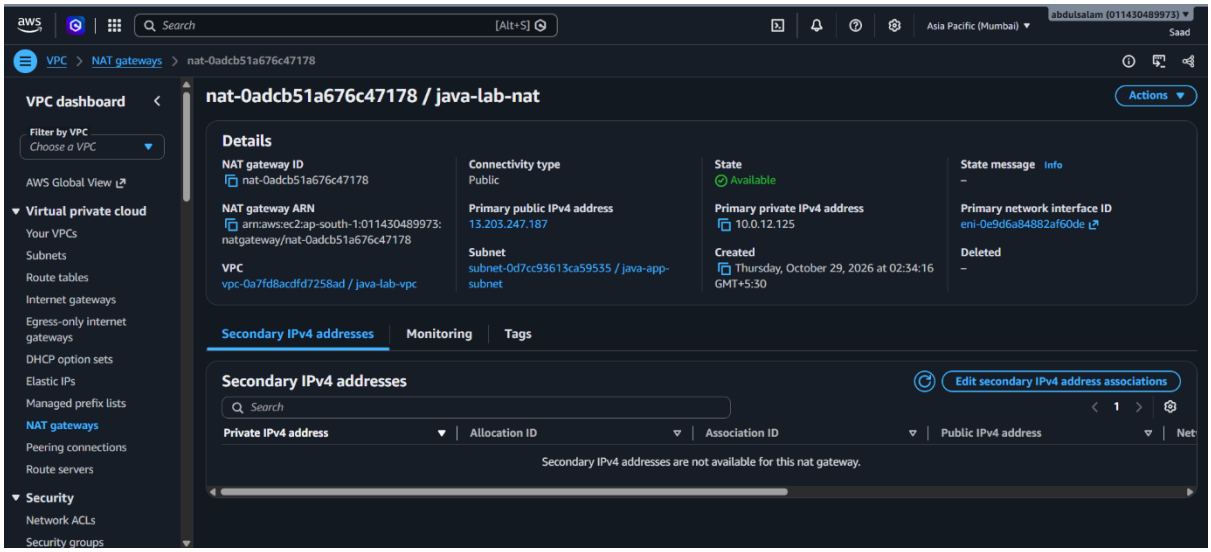
- Associated the public route table with the application subnet.
- Configured the private route table.
- Configured the NAT Gateway and the private default route as required by the lab.
- Associated the private route table with the appropriate private subnet.
- Verified the routes and subnet associations.

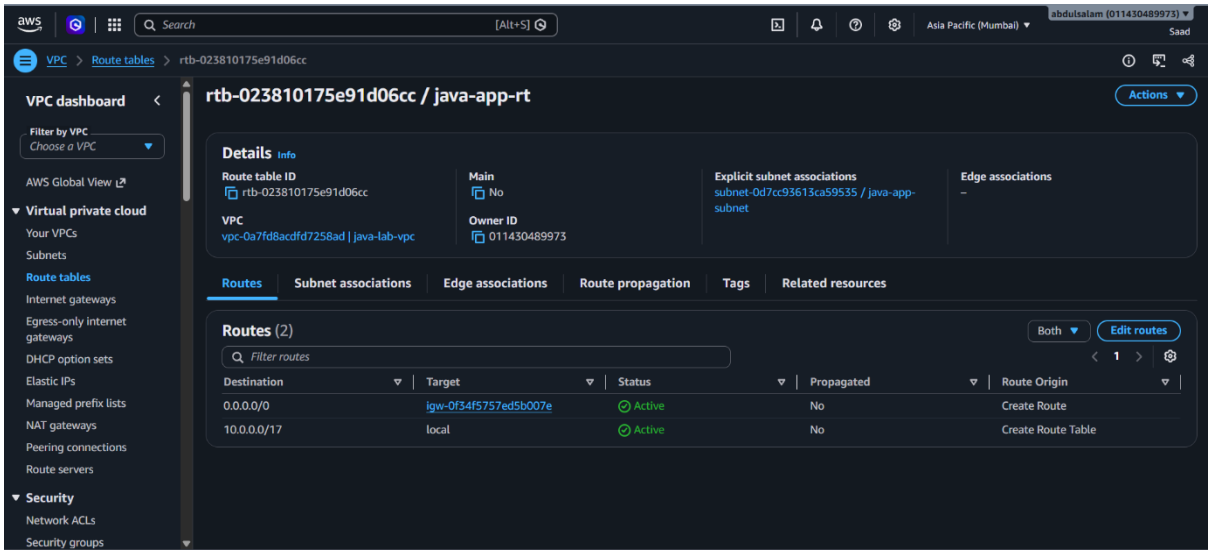
The Internet Gateway provides internet connectivity for appropriately configured public resources. The NAT Gateway allows eligible private subnet resources to initiate outbound internet connections without requiring public IPv4 addresses.

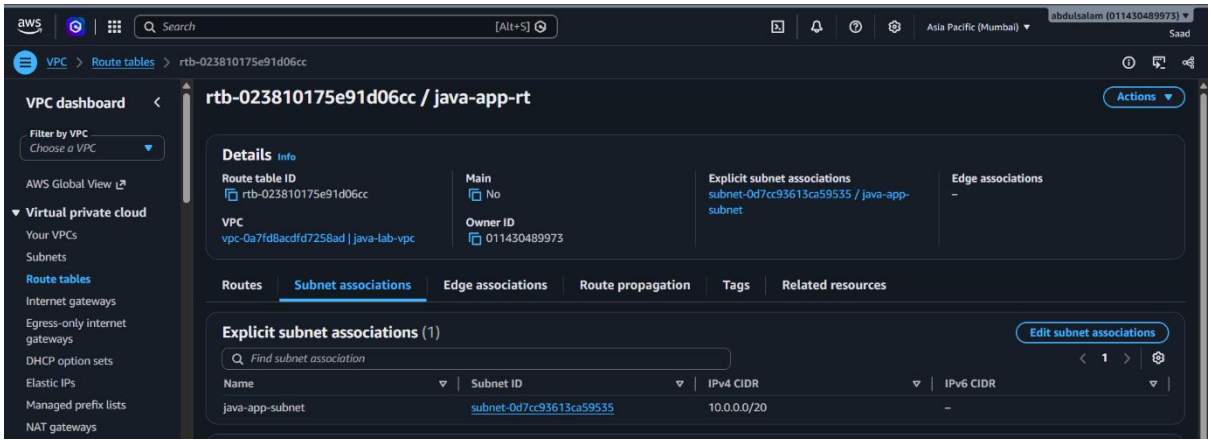
Recorded network resources:

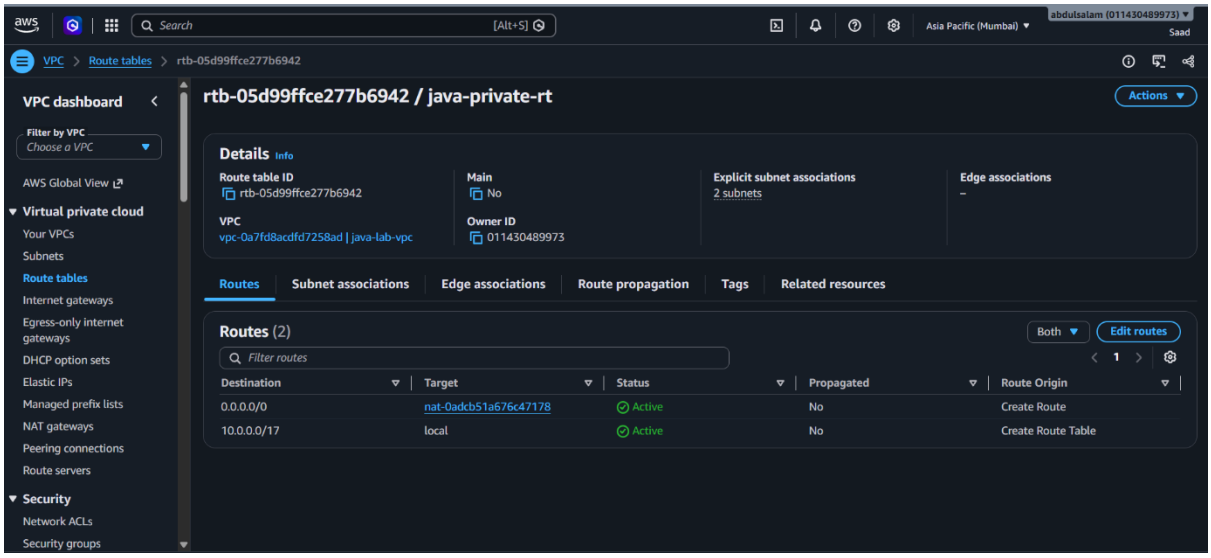
- Internet Gateway: java-lab-igw
- NAT Gateway: java-lab-nat
- Public route table: java-app-rt
- Private route table: java-private-rt




Search [Alt+S]
Asia Pacific (Mumbai)
abdulsalam (011430489973)
Saad
VPC > NAT gateways > nat-0adcb51a676c47178
VPC dashboard
Filter by VPC
Choose a VPC
AWS Global View
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
Peering connections
Route servers
Security
Network ACLs
Security groups
nat-0adcb51a676c47178 / java-lab-nat
Actions
Details
NAT gateway ID
nat-0adcb51a676c47178
Connectivity type
Public
State
Available
State message Info
–
NAT gateway ARN
arn:aws:ec2:ap-south-1:011430489973:natgateway/nat-0adcb51a676c47178
Primary public IPv4 address
13.203.247.187
Primary private IPv4 address
10.0.12.125
Primary network interface ID
eni-0e9d6a84882af60de
VPC
vpc-0a7fd8acdfd7258ad / java-lab-vpc
Subnet
subnet-0d7cc93613ca59535 / java-app-subnet
Created
Thursday, October 29, 2026 at 02:34:16 GMT+5:30
Deleted
–
Secondary IPv4 addresses
Monitoring
Tags
Secondary IPv4 addresses
Edit secondary IPv4 address associations
Search
Private IPv4 address
Allocation ID
Association ID
Public IPv4 address
Secondary IPv4 addresses are not available for this nat gateway.


Search [Alt+S]
Asia Pacific (Mumbai)
abdulsalam (011430489973)
Saad
VPC > Route tables > rtb-023810175e91d06cc
VPC dashboard
Filter by VPC
Choose a VPC
AWS Global View
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
Peering connections
Route servers
Security
Network ACLs
Security groups
rtb-023810175e91d06cc / java-app-rt
Actions
Details Info
Route table ID
rtb-023810175e91d06cc
Main
No
Explicit subnet associations
subnet-0d7cc93613ca59535 / java-app-subnet
Edge associations
–
VPC
vpc-0a7fd8acdfd7258ad | java-lab-vpc
Owner ID
011430489973
Routes
Subnet associations
Edge associations
Route propagation
Tags
Related resources
Routes (2)
Both
Edit routes
Filter routes
Destination
Target
Status
Propagated
Route Origin
0.0.0.0/0
igw-0f34f5757ed5b007e
Active
No
Create Route
10.0.0.0/17
local
Active
No
Create Route Table


Search [Alt+S]
Asia Pacific (Mumbai)
abdulsalam (011430489973)
Saad
VPC > Route tables > rtb-023810175e91d06cc
VPC dashboard
Filter by VPC
Choose a VPC
AWS Global View
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
rtb-023810175e91d06cc / java-app-rt
Actions
Details Info
Route table ID
rtb-023810175e91d06cc
Main
No
Explicit subnet associations
subnet-0d7cc93613ca59535 / java-app-subnet
Edge associations
–
VPC
vpc-0a7fd8acdfd7258ad | java-lab-vpc
Owner ID
011430489973
Routes
Subnet associations
Edge associations
Route propagation
Tags
Related resources
Explicit subnet associations (1)
Edit subnet associations
Find subnet association
Name
Subnet ID
IPv4 CIDR
IPv6 CIDR
java-app-subnet
subnet-0d7cc93613ca59535
10.0.0.0/20
–



4. Amazon RDS Configuration

4.1 Creating the RDS Database

I created an Amazon RDS MySQL database instance named java-lab-mysql.

The RDS database is used to store the registration information of the Java web application.

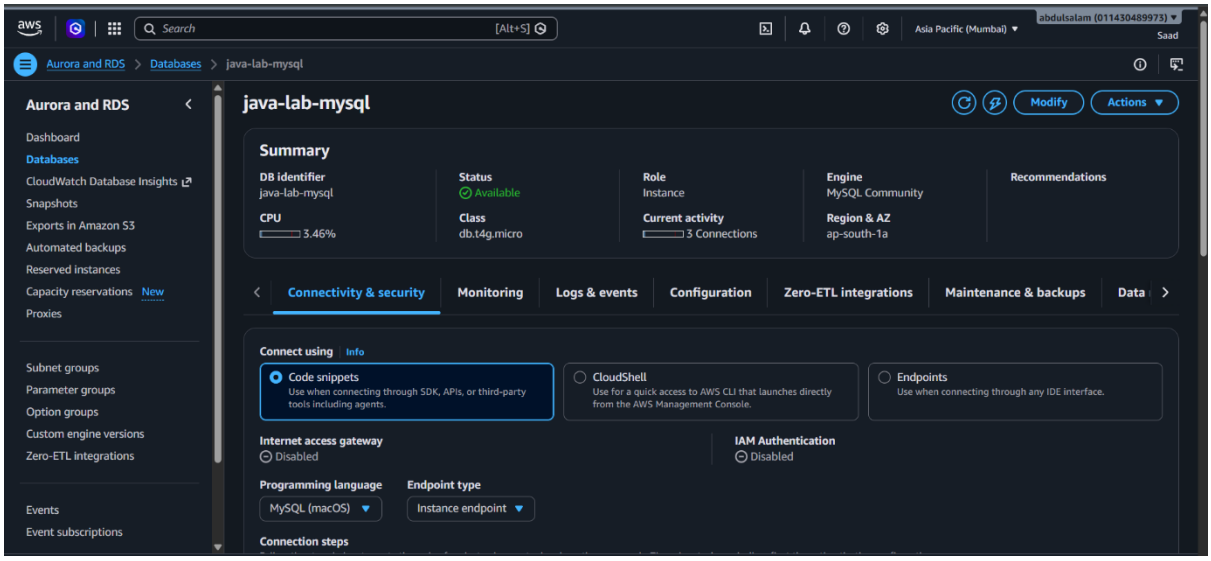
Recorded configuration:

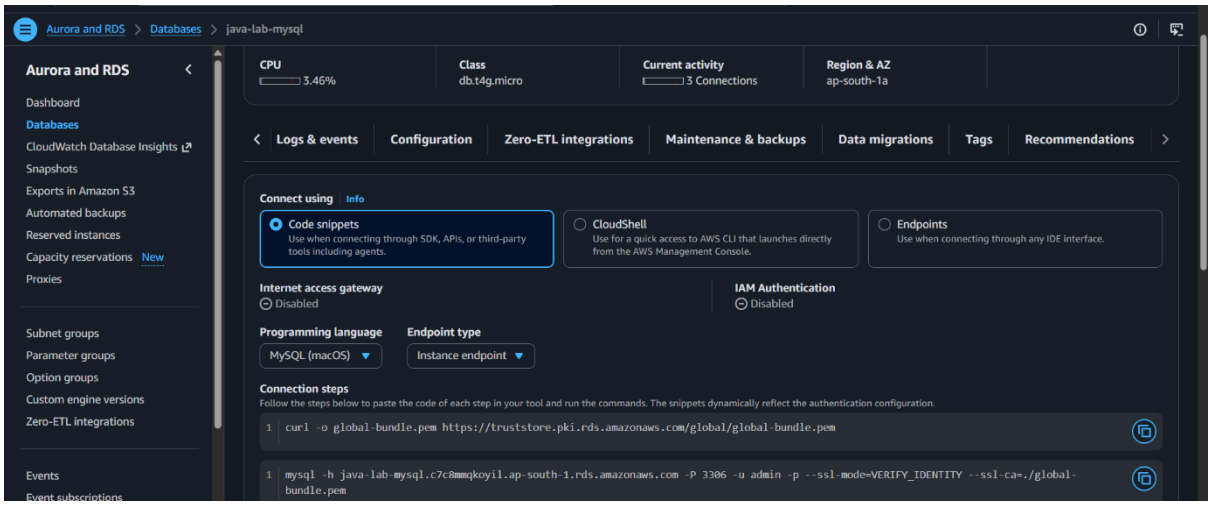
- DB identifier: java-lab-mysql
- Engine: MySQL
- Endpoint: java-lab-mysql.c7c8mmqkoyil.ap-south-1.rds.amazonaws.com
- Port: 3306
- Database name used by the application: jwt
- VPC: java-lab-vpc

Procedure:

- Opened Amazon RDS.
- Selected Databases and Create database.

- Selected the MySQL database engine.
- Configured the database identifier and administrator credentials.
- Selected the appropriate DB instance and storage configuration.
- Selected the DB subnet group and VPC according to the lab.
- Configured database connectivity and security settings.
- Created the database instance.
- Waited until the database status became Available.
- Recorded the database endpoint and port.







4.2 Configuring Database Security

I configured the security group associated with RDS to restrict database access.

The RDS security group used in the lab was:

java-rds-mysql-sg

Security group ID:

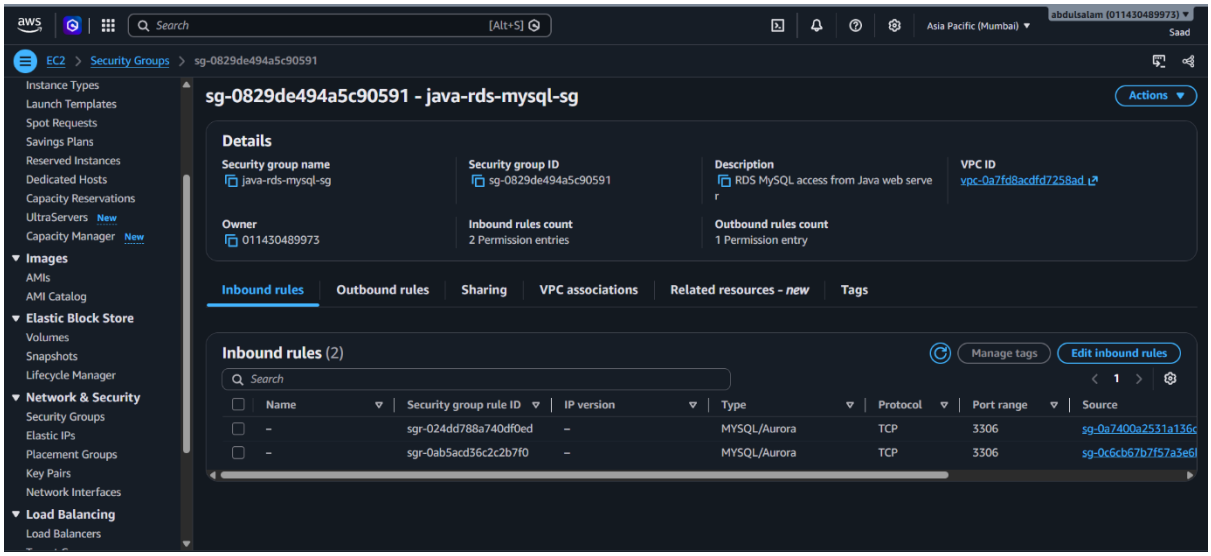
sg-0829de494a5c90591

MySQL uses TCP port 3306.

The intended database access rules allowed connections from the application server security group and the private VM security group.

Procedure:

- Opened EC2 → Security Groups.
- Selected java-rds-mysql-sg.
- Opened Inbound rules.
- Reviewed the MySQL/Aurora TCP 3306 rule.
- Allowed the required source security groups.
- Saved the rules and verified the configuration.



5. Server Management

5.1 Configuring Server 1 – Application Server

I launched an EC2 instance named java-app-server to host the Java web application.

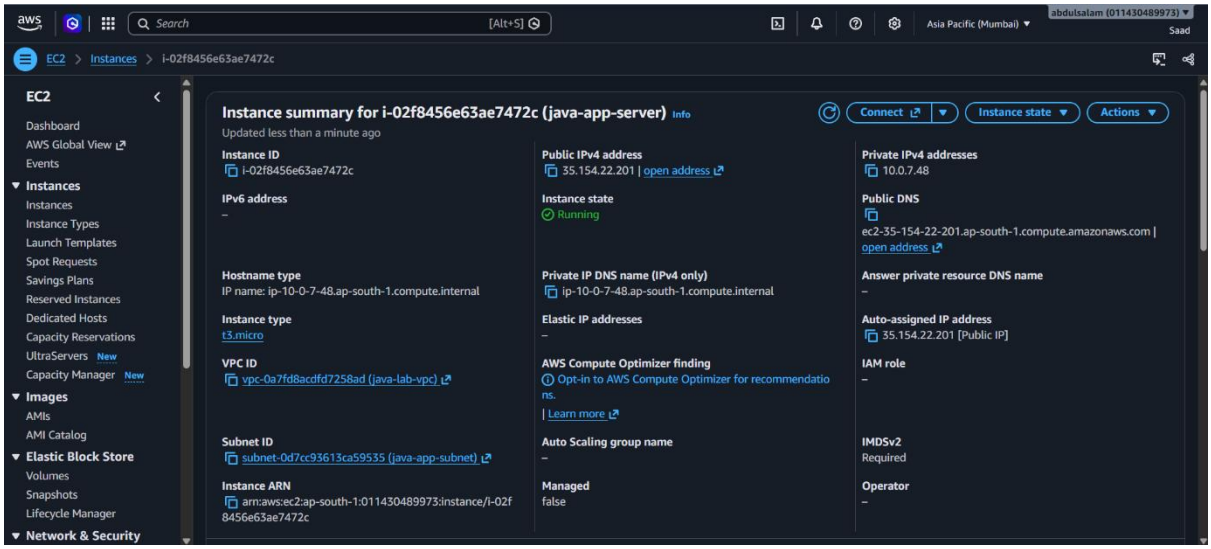
Recorded details:

- Private IPv4: 10.0.7.48
- Public IPv4 used during testing: 35.154.22.201
- Operating system: Ubuntu

Procedure:

- Opened EC2 → Instances.
- Selected Launch instance.
- Entered the instance name.
- Selected the Ubuntu AMI.
- Selected the required instance type and key pair.
- Selected the custom VPC and application subnet.
- Configured the security group.

- Launched the instance.
- Waited for the instance to become running.
- Connected to the instance using SSH.



5.2 Installing Application Software

I installed the software required to build and deploy the application.

The following tools were installed:

- OpenJDK 17
- Git
- Apache Maven
- Apache Tomcat 9

I verified the Java, Git, and Maven versions after installation.

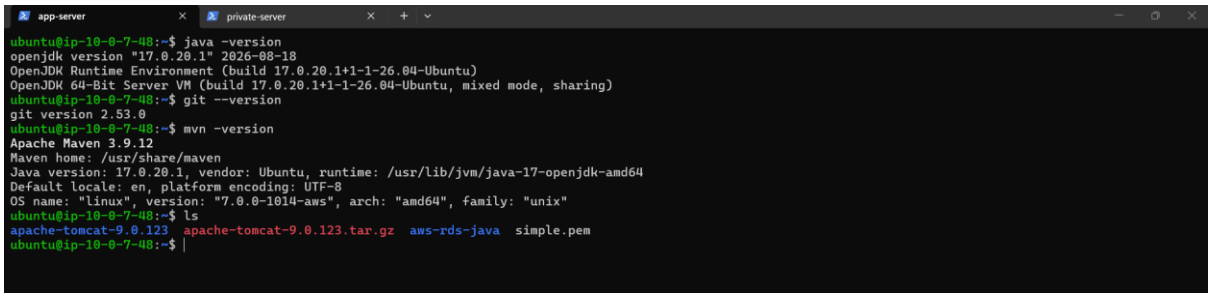
Commands used for validation included:

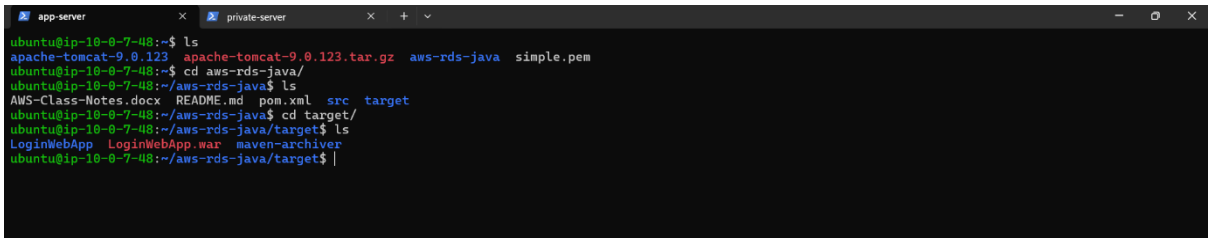
java -version

git --version

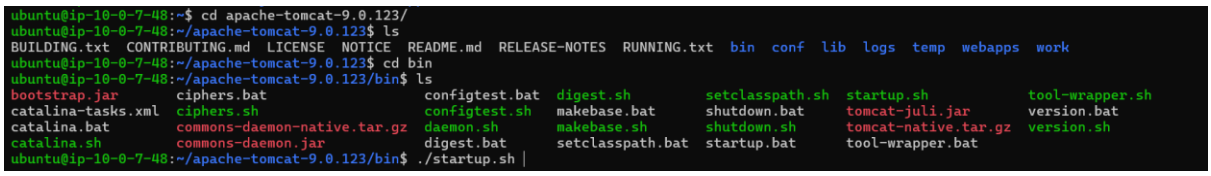
mvn -version

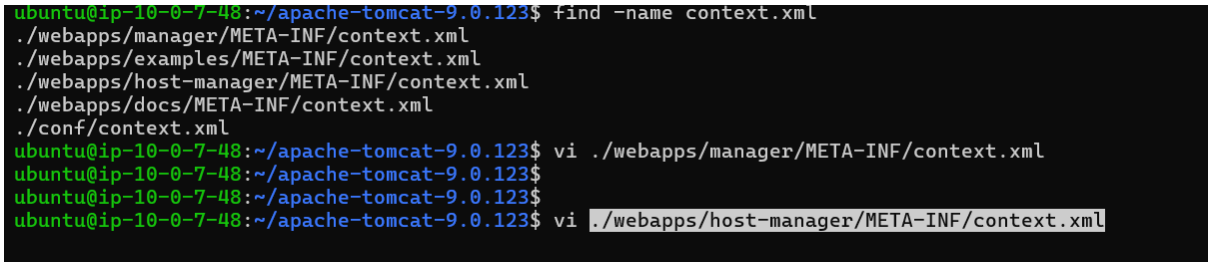
Apache Tomcat was installed under /opt/tomcat and configured to run using a dedicated service user.

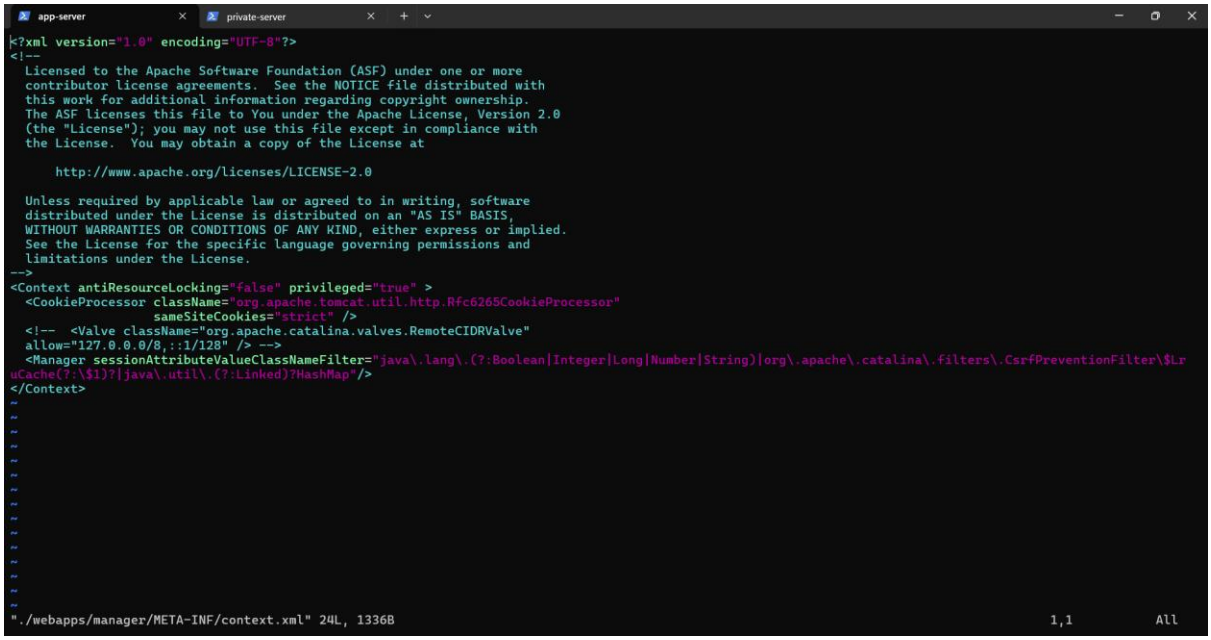


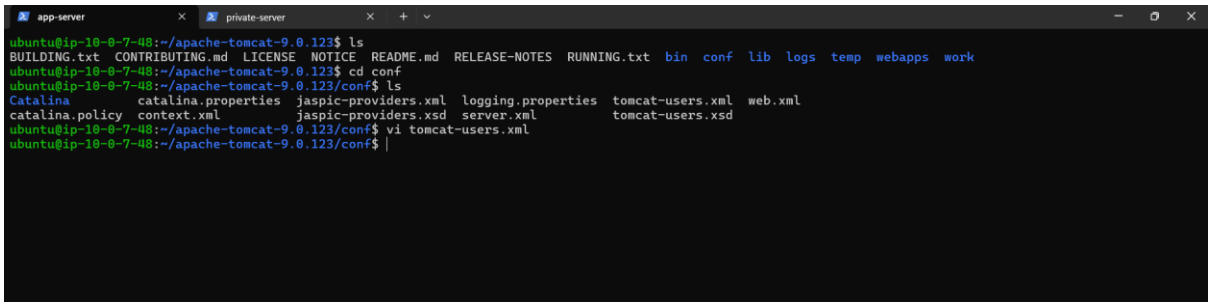


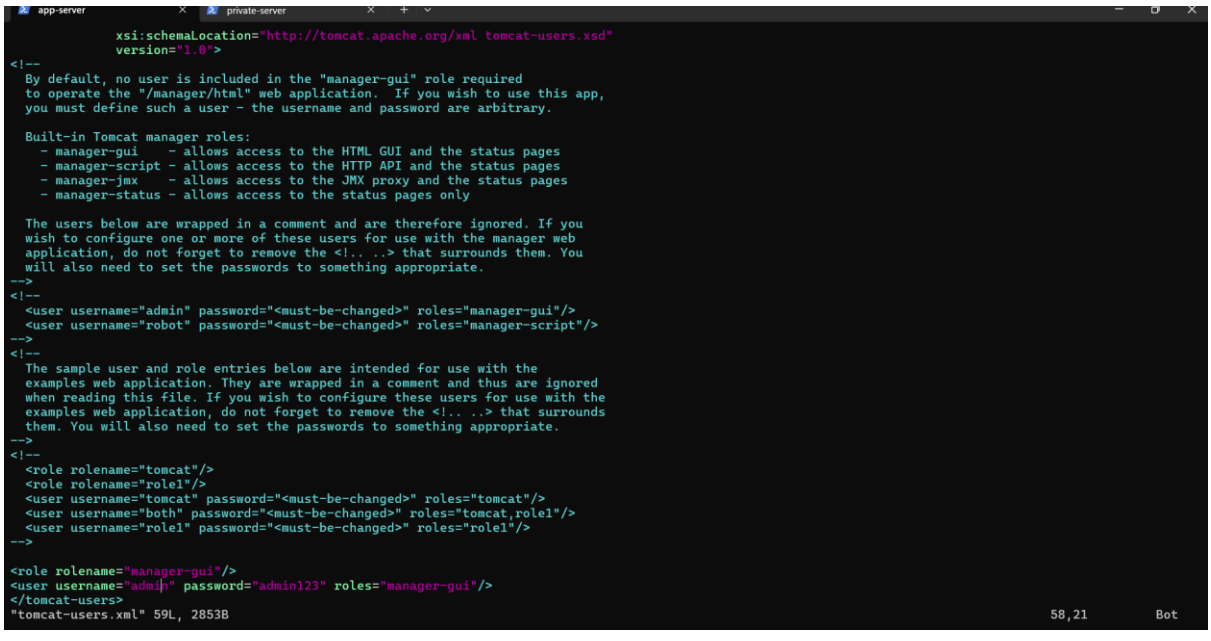







app-server
private-server
<?xml version="1.0" encoding="UTF-8"?>
<!--
Licensed to the Apache Software Foundation (ASF) under one or more
contributor license agreements. See the NOTICE file distributed with
this work for additional information regarding copyright ownership.
The ASF licenses this file to You under the Apache License, Version 2.0
(the "License"); you may not use this file except in compliance with
the License. You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
-->
<Context antiResourceLocking="false" privileged="true" >
<CookieProcessor className="org.apache.tomcat.util.http.Rfc6265CookieProcessor"
sameSiteCookies="strict" />
<!-- <Valve className="org.apache.catalina.valves.RemoteCIDRValve"
allow="127.0.0.0/8,::1/128" /> -->
<Manager sessionAttributeValueClassNameFilter="java\.lang\.(?:Boolean|Integer|Long|Number|String)|org\.apache\.catalina\.filters\.CsrfPreventionFilter\$LruCache(?:\$1)?|java\.util\.(?:Linked)?HashMap"/>
</Context>
"./webapps/manager/META-INF/context.xml" 24L, 1336B
1,1
All


app-server
private-server
ubuntu@ip-10-0-7-48:~/apache-tomcat-9.0.123$ ls
BUILDING.txt CONTRIBUTING.md LICENSE NOTICE README.md RELEASE-NOTES RUNNING.txt bin conf lib logs temp webapps work
ubuntu@ip-10-0-7-48:~/apache-tomcat-9.0.123$ cd conf
ubuntu@ip-10-0-7-48:~/apache-tomcat-9.0.123/conf$ ls
Catalina catalina.properties jaspic-providers.xml logging.properties tomcat-users.xml web.xml
catalina.policy context.xml jaspic-providers.xsd server.xml tomcat-users.xsd
ubuntu@ip-10-0-7-48:~/apache-tomcat-9.0.123/conf$ vi tomcat-users.xml
ubuntu@ip-10-0-7-48:~/apache-tomcat-9.0.123/conf$


app-server
private-server
xsi:schemaLocation="http://tomcat.apache.org/xml tomcat-users.xsd"
version="1.0">
<!--
By default, no user is included in the "manager-gui" role required
to operate the "/manager/html" web application. If you wish to use this app,
you must define such a user - the username and password are arbitrary.
Built-in Tomcat manager roles:
- manager-gui - allows access to the HTML GUI and the status pages
- manager-script - allows access to the HTTP API and the status pages
- manager-jmx - allows access to the JMX proxy and the status pages
- manager-status - allows access to the status pages only
The users below are wrapped in a comment and are therefore ignored. If you
wish to configure one or more of these users for use with the manager web
application, do not forget to remove the <!.. ..> that surrounds them. You
will also need to set the passwords to something appropriate.
-->
<!--
<user username="admin" password="<must-be-changed>" roles="manager-gui"/>
<user username="robot" password="<must-be-changed>" roles="manager-script"/>
-->
<!--
The sample user and role entries below are intended for use with the
examples web application. They are wrapped in a comment and thus are ignored
when reading this file. If you wish to configure these users for use with the
examples web application, do not forget to remove the <!.. ..> that surrounds
them. You will also need to set the passwords to something appropriate.
-->
<!--
<role rolename="tomcat"/>
<role rolename="role1"/>
<user username="tomcat" password="<must-be-changed>" roles="tomcat"/>
<user username="both" password="<must-be-changed>" roles="tomcat,role1"/>
<user username="role1" password="<must-be-changed>" roles="role1"/>
-->
<role rolename="manager-gui"/>
<user username="admin" password="admin123" roles="manager-gui"/>
</tomcat-users>
"tomcat-users.xml" 59L, 2853B
58,21
Bot

5.3 Configuring Server 2 – Private Virtual Machine

I created a private EC2 virtual machine named java-private-vm.

The private VM is used for database administration and connecting to Amazon RDS.

Recorded private IP: 10.0.16.41

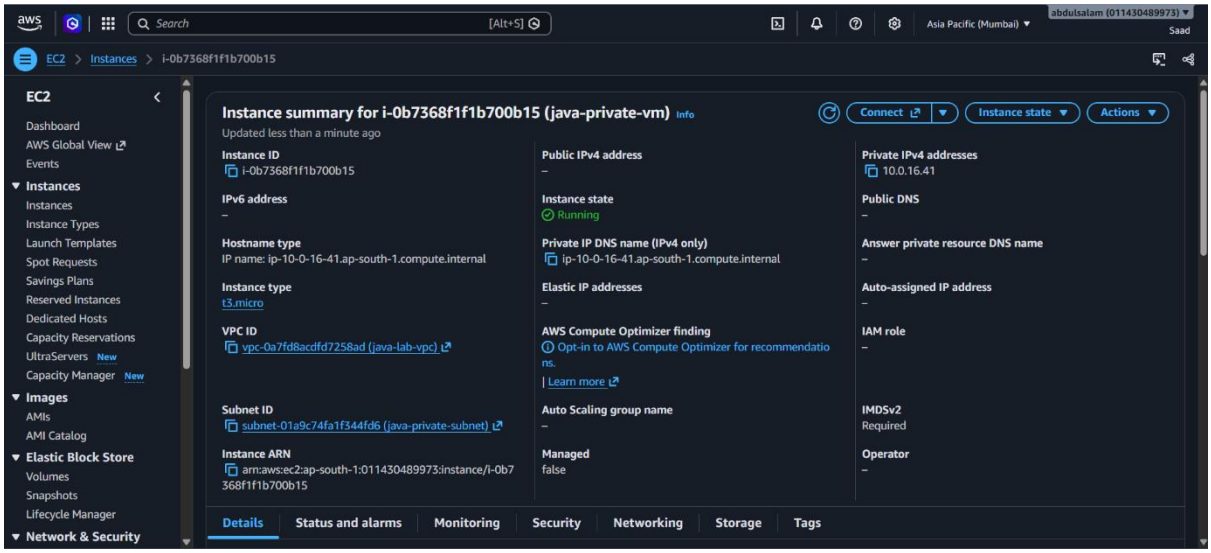
Procedure:

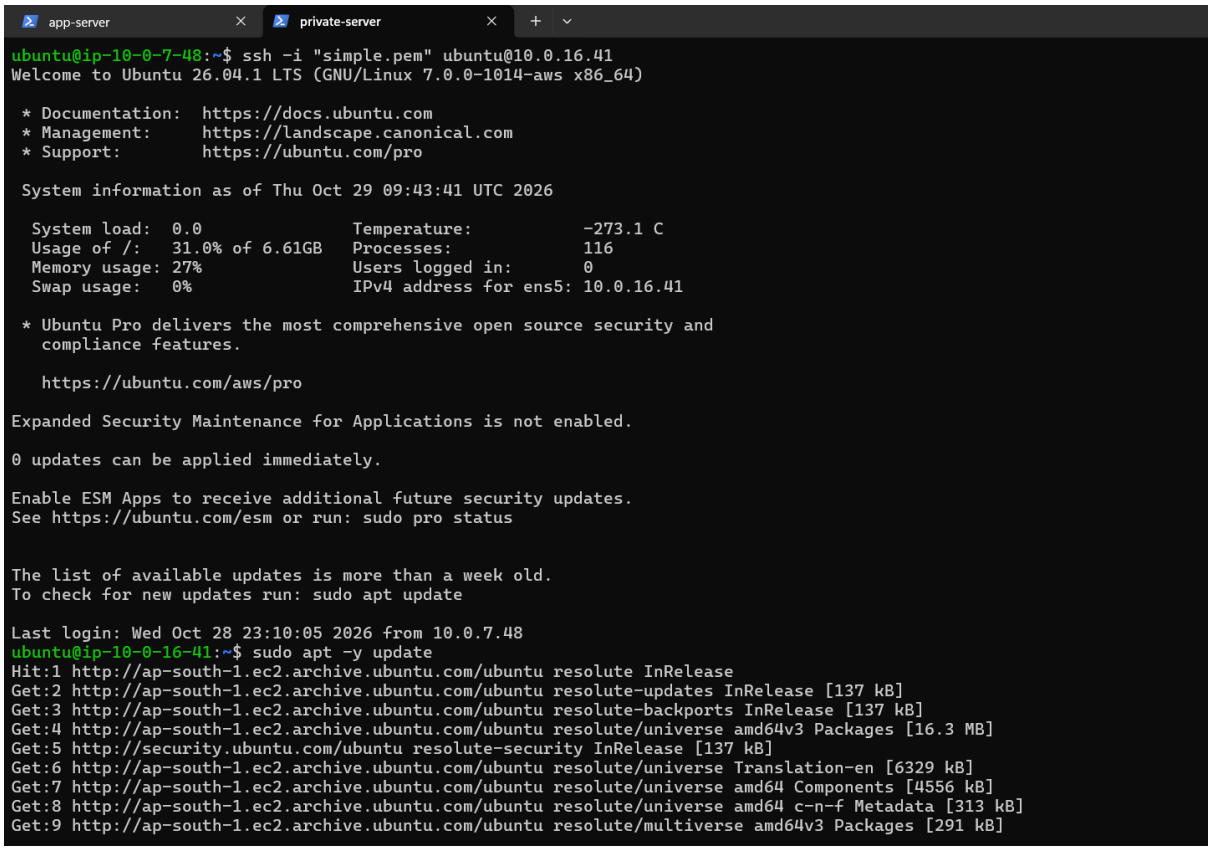
- Opened EC2 and selected Launch instance.
- Selected the Ubuntu AMI.
- Configured the instance name and key pair.
- Selected java-lab-vpc.
- Selected the private subnet java-private-subnet.
- Disabled public IPv4 address assignment.
- Attached the private VM security group.
- Launched the instance.
- Verified the instance state and private IP address.

I connected from the application server to the private VM using SSH.

ssh -i "simple.pem" ubuntu@10.0.16.41

The SSH key permissions were corrected after an initial authentication problem.





6. Connecting the Private VM to Amazon RDS

I used the MySQL client on the private VM to connect to the Amazon RDS database.

The database endpoint was:

java-lab-mysql.c7c8mmqkoyil.ap-south-1.rds.amazonaws.com

The connection command was:

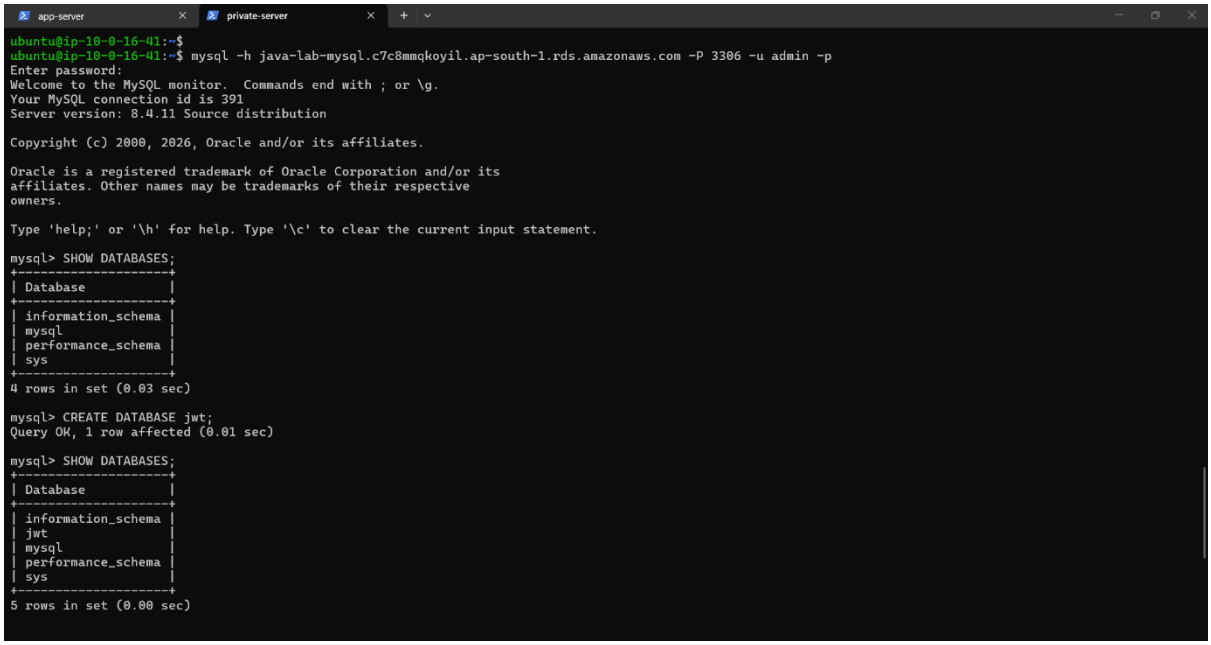
mysql -h java-lab-mysql.c7c8mmqkoyil.ap-south-1.rds.amazonaws.com \

-P 3306 -u admin -p

After entering the database password, the MySQL client connected successfully.

The MySQL server reported version 8.4.11.

This verified that the private VM could communicate with the RDS database.



7. Creating the Database Schema

After connecting to RDS, I verified the available databases.

SHOW DATABASES;

The jwt database was available.

I selected the database:

USE jwt;

I checked the existing tables:

SHOW TABLES;

The USER table was then created using the application's required columns and constraints.

The table included:

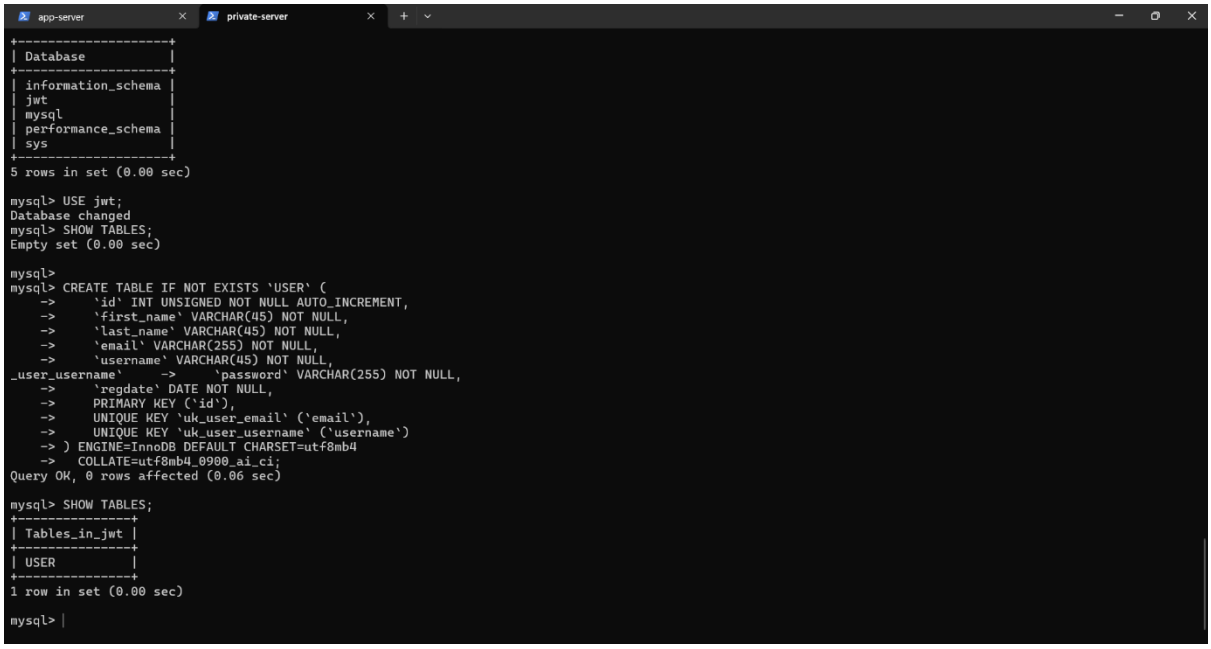
- id
- first_name
- last_name
- email
- username
- password
- regdate

The table used a primary key and unique constraints for email and username.

I verified the table creation:

SHOW TABLES;

DESCRIBE `USER`;



8. Building the Java Application

I used the existing Java web application project from GitHub.

Repository:

https://github.com/Akiranred/aws-rds-java

I cloned the project to the application server.

git clone https://github.com/Akiranred/aws-rds-java.git

cd aws-rds-java

I reviewed the project files and configured the application's database connection settings to use Amazon RDS and the jwt database.

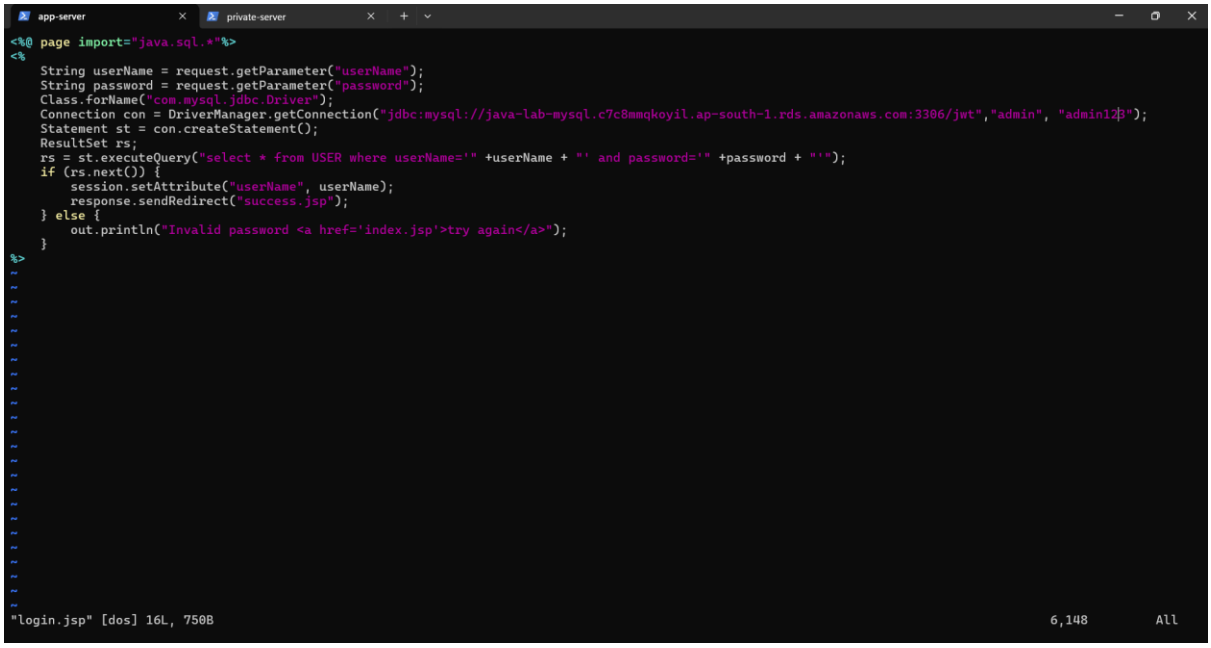
I built the application using Maven:

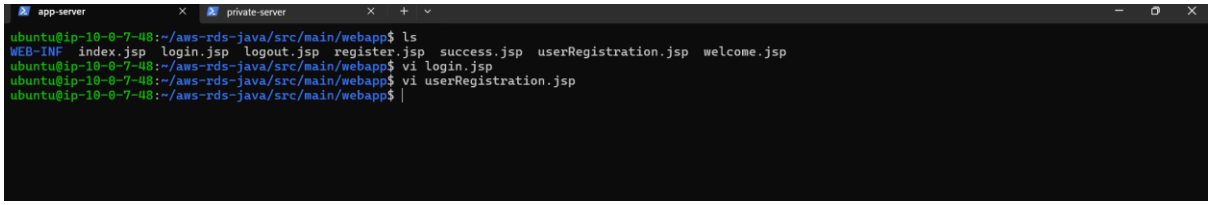
mvn clean package

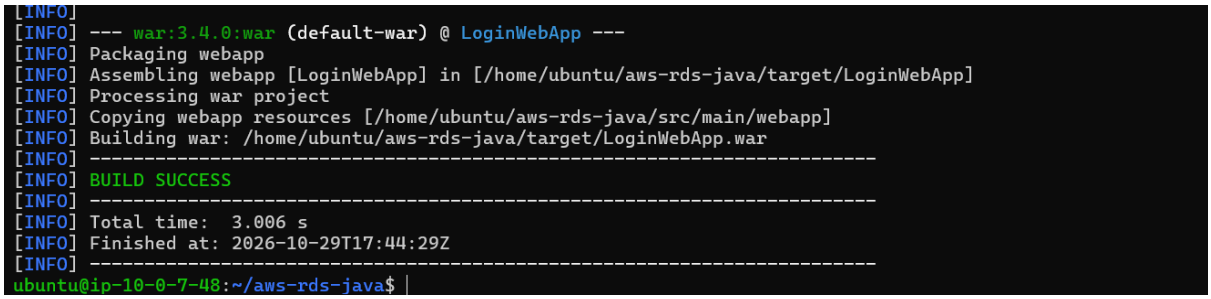
The build generated the following WAR artifact:

target/LoginWebApp.war

```
ubuntu@ip-10-0-7-48:~/aws-rds-java$ cat pom.xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.javawebtutor</groupId>
  <artifactId>LoginWebApp</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>war</packaging>
  <name>LoginWebApp</name>
  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>9.5.0</version>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <finalName>LoginWebApp</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.14.1</version>
```









9. Deploying the Application on Apache Tomcat

I deployed the generated WAR file to Apache Tomcat.

The deployment artifact was copied into Tomcat's webapps directory.

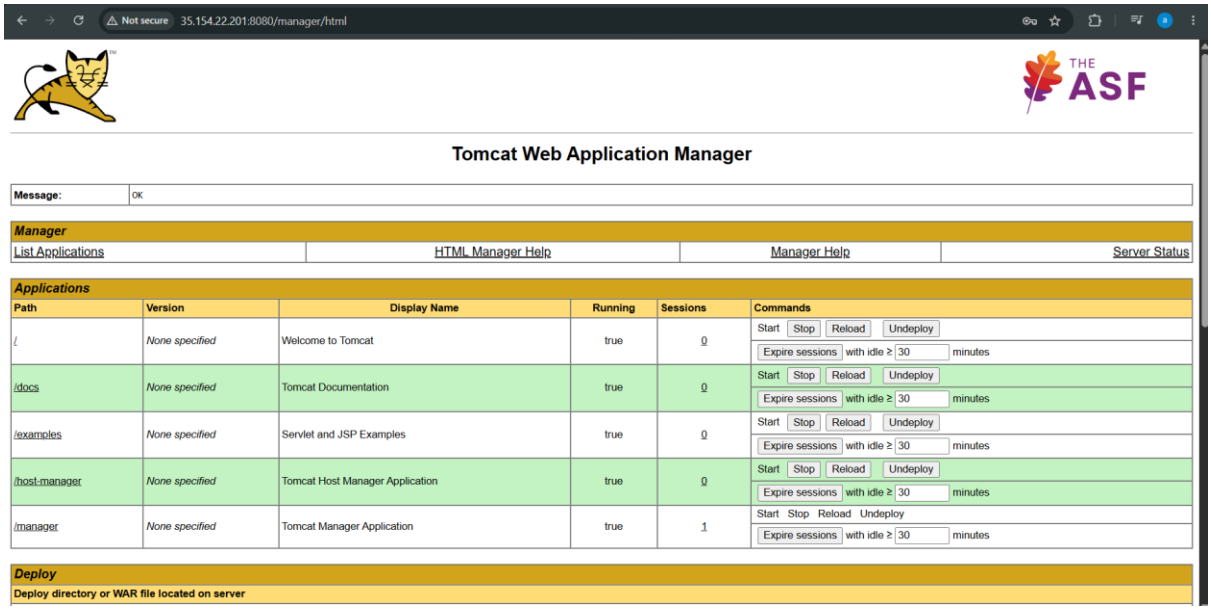
The deployed application context was:

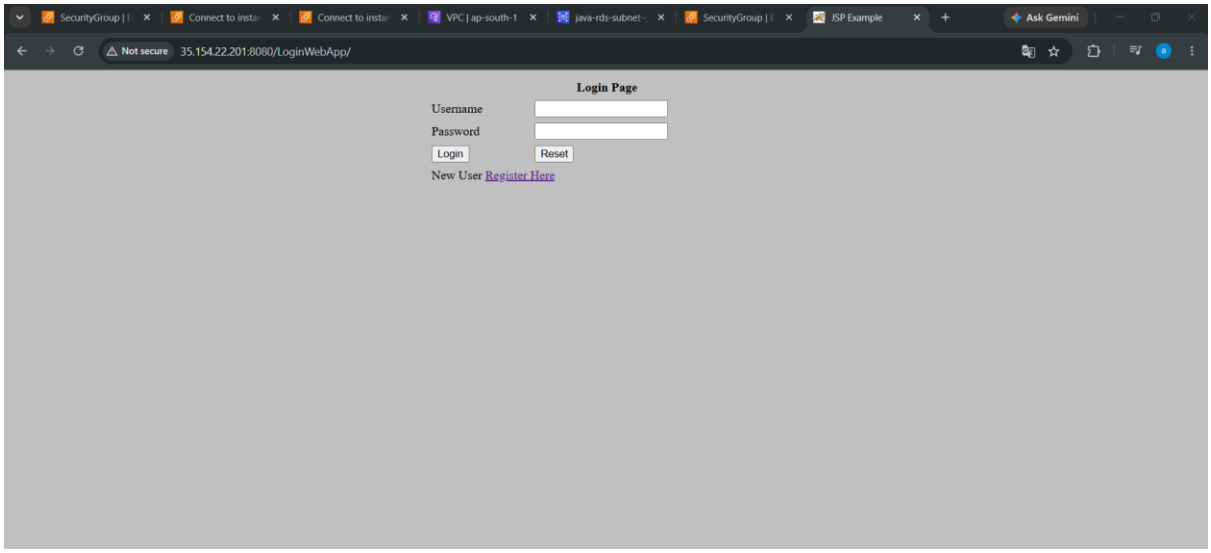
/LoginWebApp

I verified that Tomcat was running and accessed the application using a browser.

Application URL used during testing:

http://35.154.22.201:8080/LoginWebApp/





10. Application Testing

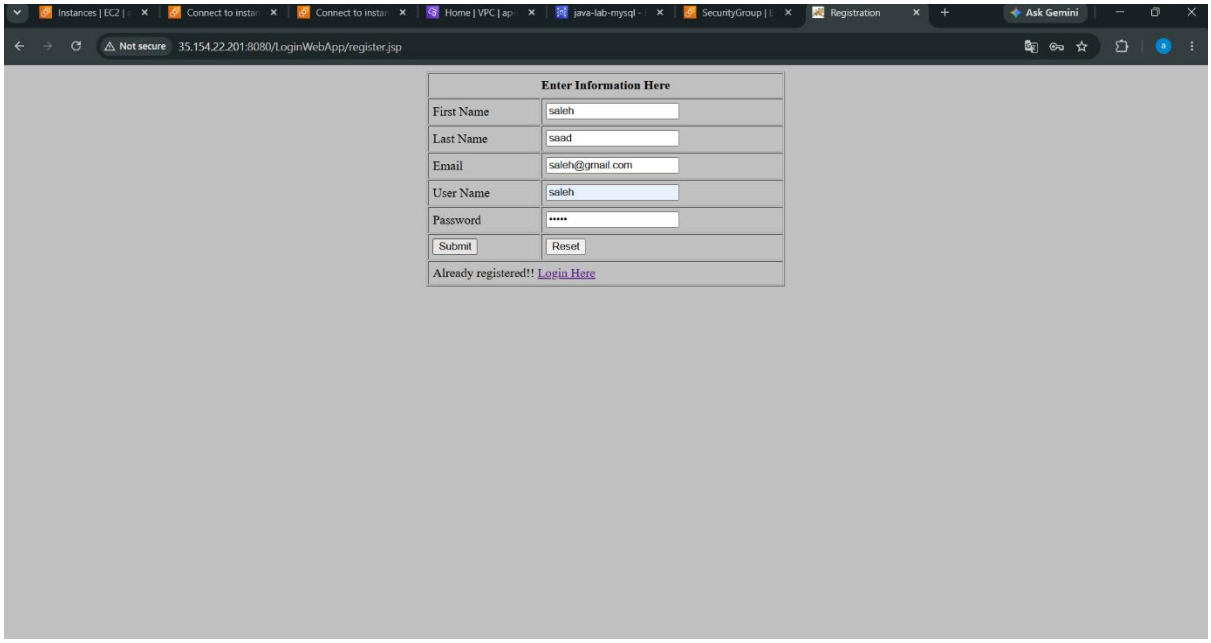
10.1 Registration Test

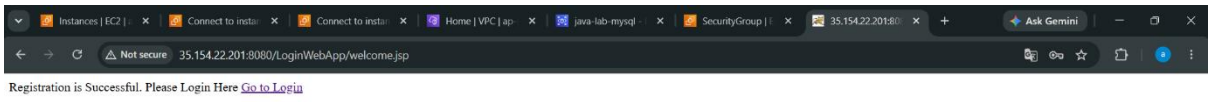
I opened the registration page and entered dummy test-user information.

After submitting the form, the application displayed:

“Registration is Successful.”

I then verified the database records in RDS.





10.2 Database Record Verification

I executed the following SQL commands:

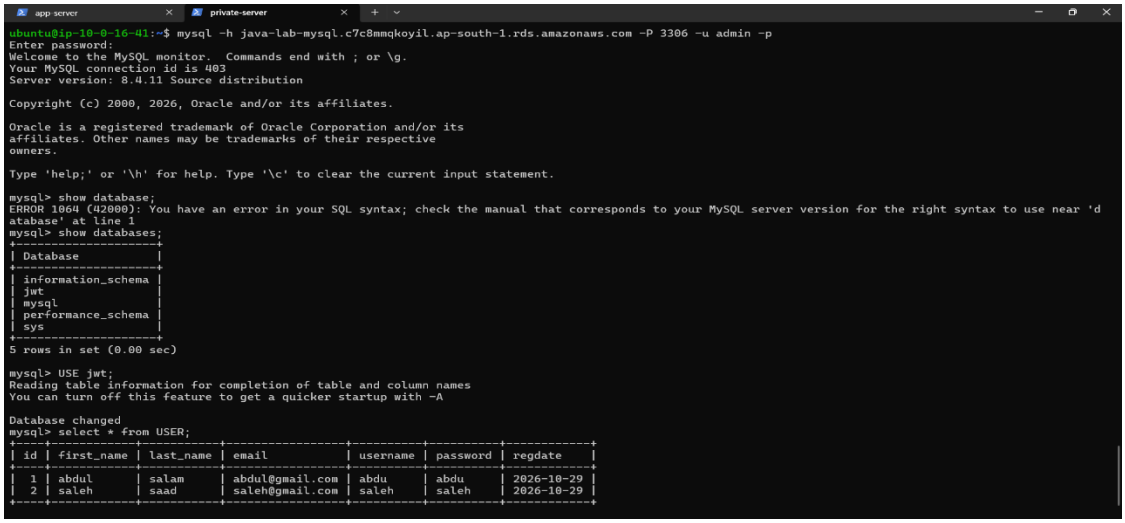
USE jwt;

SELECT * FROM USER;

The query displayed the registered test users.

This confirmed that registration data was saved in the RDS database.

For the submitted report, sensitive password values should be hidden or excluded from screenshots.



10.3 Login Test

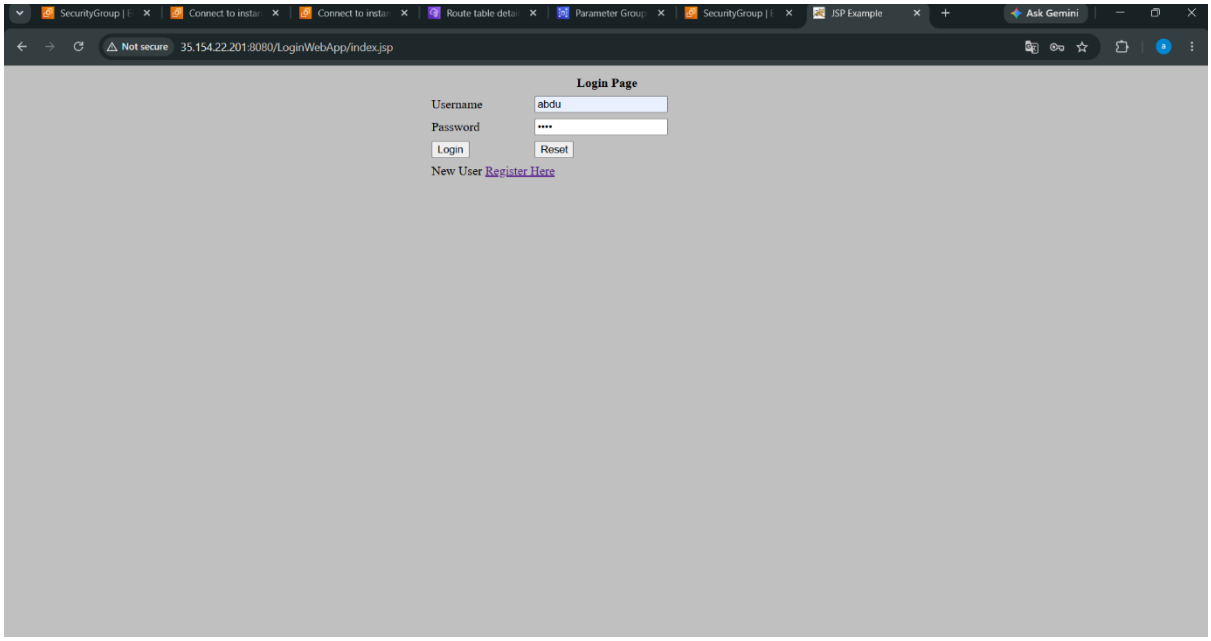
I returned to the application's login page and entered the credentials for a test account.

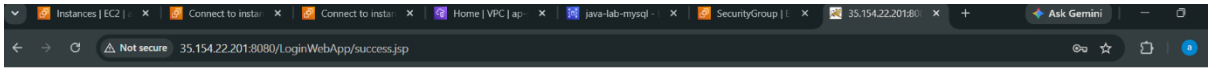
After successful authentication, the application displayed the welcome page.

The browser showed:

“Welcome abdul”

This verified the login workflow for the tested account.





Welcome abdu Log out

12. Conclusion

This project demonstrated the deployment of a Java login and registration web application on AWS using Amazon EC2, Apache Tomcat, Maven, and Amazon RDS for MySQL.

The infrastructure was configured using a custom VPC with public and private subnets. A private virtual machine was used for database administration, and the application was deployed on the public application server.

Registration and login tests were performed successfully, and the registered records were verified in RDS.

The lab provided practical experience in AWS networking, server management, database connectivity, application deployment, and cloud security configuration.